Smartly: Ad Campaign DataFlow

Apache Airflow

dbt

docker

Google BigQuery

SQL

python™

Contents

1) Project Title 3
2) Objectives / Problem Statement 3
3) Tech Stack 4
4) Architecture / System Design 5
Workflow Steps: 5
5) Implementation Steps 6
1. Data Ingestion (Python) 6
2. Data Quality Checks (Python) 6
3. Transformation and Modeling (DBT) 6
4. Testing (DBT) 7
5. Orchestration (Airflow) 7
6) Code and Results 8
7) Summary 16

1) Project Title

- **SMARTLY : Ad Campaign DataFlow**

2) Objectives / Problem Statement

- A marketing analytics company runs multiple advertising campaigns on Facebook. Raw campaign performance data is collected as CSV exports that include metrics such as impressions, clicks, spend and conversions.

- Dataset - https://www.kaggle.com/datasets/loveall/clicks-conversion-tracking

❖ *The problem:*
- Raw CSVs are **inconsistent**, **not cleaned** and **don't provide analytical insights** directly.

❖ *Goal*:

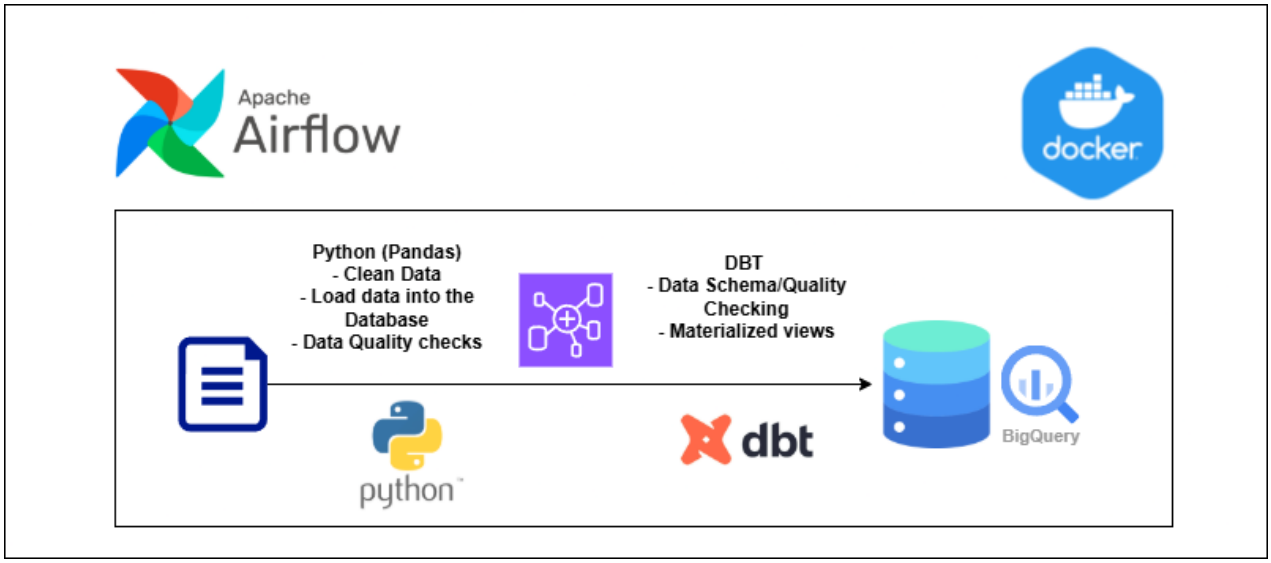
- Build a production-grade data pipeline that:
  - Ingests raw Facebook Ad campaign CSVs automatically.
  - Cleans, validates, and loads data into a centralized data warehouse (GCP BigQuery).
  - Applies modular data transformations and testing with DBT.
  - Automates end-to-end orchestration using Apache Airflow (Docker-based).
  - Produces analytics-ready models and materialized views for business reporting

3) Tech Stack

Layer	Tool	Purpose
Data Source	CSV File	Raw Facebook ad campaign data
Ingestion & Cleaning	Python (Pandas, SQLAlchemy, GCP SDK)	Extracts CSV → cleans → loads to BigQuery
Storage / Warehouse	GCP BigQuery (previously PostgreSQL)	Centralized data warehouse
Transformation & Modeling	dbt (Data Build Tool)	Data transformations, modeling and testing
Orchestration	Apache Airflow (Docker)	Automates pipeline execution (daily at 12 AM)
Version Control	Git / GitHub	Code management and collaboration
Environment Management	Python dotenv	Secure configuration and secret management

4) Architecture / System Design

- Pipeline Flow



CSV Source >> Data Load (Python) >> Data Quality Check >> DBT Modeling >> DBT Tests >> Data Load to Model Table >> DBT Modeling Metrics >> DBT Tests

Workflow Steps:

- Airflow DAG orchestrates all pipeline tasks in Dockerized environment.
- Python scripts handle ingestion, validation and data loading into BigQuery.
- dbt models transform, clean and aggregate data into analytics ready tables.
- dbt tests ensure data integrity and quality.
- Final materialized views support reporting dashboards or BI tools.

5) Implementation Steps

1. Data Ingestion (Python)

- Read raw CSVs with Pandas.
- Standardize column names and handle nulls.
- Validate schema and filter out invalid records (ex: negative spend or clicks).
- Load cleaned data into BigQuery from raw_ad_logs.csv

2. Data Quality Checks (Python)

- Custom Python script run_quality_checks_staging_table.py:
    - Checks for nulls
    - Validates non-negative numeric columns
    - Detects duplicates
    - Logs all results to logs/ run_quality_checks_staging_table.log

3. Transformation and Modeling (DBT)

- DBT Models
    - Staging
        - daily_ad_logs_view.sql (staging model)
          Cleans and standardizes raw data (lowercases gender, fills nulls, etc.)
        - campaign_daily_metrics_view.sql
          Aggregate data and group by company_id
    - Data_model
        - campaign_metrics_view.sql
          aggregate all the metrics for whole the period

- Data Model Table
    - Aggregate columns and derived new values
        - Click_rate = clicks / impressions
        - Spent_rate = spend / impressions
        - Conversion_rate = conversions / clicks

4. Testing (DBT)

- schema.yml defines data quality tests:
  - not_null, unique, accepted_values, dbt_utils.expression_is_true
- Custom test macros for non_negative and logical data validation
- Tests run automatically with:
  - dbt test with Airflow automation

5. Orchestration (Airflow)

- DAG Schedule: Daily at 12:00 AM

Execution order:

- load_to_bigquery.py - Ingests CSVs to BigQuery.
- run_quality_checks_staging_table.py - Validates staging layer.
- dbt models:
  - daily_ad_logs_view.sql - Build staging materialized views.
  - Campaign_daily_metrics_view.sql – Build metrics in staging materialized views

- dbt tests - Validate staging layer.
- staging_to_datamodel.py - Move staging to model tables.
- dbt model:
  - campaign_daily_metrics.sql - Final analytics view.
- dbt tests - Validate data model layer

6) Code and Results

- *load_to_bigquery.py* – Load data into GCP BigQuery database

load_to_bigquery.py ×

scripts > load_to_bigquery.py > load_to_bigquery

```python
import os
import sys
import logging
import pandas as pd
from dotenv import load_dotenv
from google.cloud import bigquery

# setup logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("logs/load_to_bigquery.log"),
        logging.StreamHandler(sys.stdout)
    ]
)


# load environment
load_dotenv("/opt/airflow/AD-CAMPAIGN-DATAFLOW/.env")

CSV_PATH = os.getenv("CSV_PATH")
BQ_PROJECT = os.getenv("BQ_PROJECT")
BQ_DATASET = os.getenv("BQ_DATASET")
BQ_STG_TABLE = os.getenv("BQ_STG_TABLE")



# create BigQuery client
def get_bq_client():
    try:
        client = bigquery.Client(project=BQ_PROJECT)
        logging.info(f"BigQuery client created for project: {BQ_PROJECT}")
        return client
    except Exception as e:
        logging.error(f"Failed to create BigQuery client: {e}")
        sys.exit(1)


# read the csv data
def load_csv_data(path: str) -> pd.DataFrame:
    try:
        df = pd.read_csv(path)
        logging.info(f"Loaded {len(df)} rows from CSV file: {path}")
        return df
    except Exception as e:
        logging.error(f"Error reading CSV file: {e}")
        sys.exit(1)
```

- Result:

stg_ad_... tbl

stg_ad_logs_tbl Query Open in Share Copy Snapshot Delete Export

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

Row	ad_id	xyz_campaig...	fb_campaign...	age	gender	interest	impressions	clicks	spent	total_conver...	approved_co...
1	708746	916	103916	30-34	m	15	7350	1	1.429999948	2	1
2	708749	916	103917	30-34	m	16	17861	2	1.820000023	2	0
3	708771	916	103920	30-34	m	20	693	0	0.0	1	0
4	708815	916	103928	30-34	m	28	4259	1	1.25	1	0
5	708818	916	103928	30-34	m	28	4133	1	1.289999962	1	1
6	708820	916	103929	30-34	m	29	1915	0	0.0	1	1
7	708889	916	103940	30-34	m	15	15615	3	4.769999981	1	0
8	708895	916	103941	30-34	m	16	10951	1	1.269999981	1	1
9	708953	916	103951	30-34	m	27	2355	1	1.5	1	0
10	708958	916	103952	30-34	m	28	9502	3	3.159999967	1	0
11	708979	916	103955	30-34	m	31	1224	0	0.0	1	0
12	709023	916	103962	30-34	m	7	735	0	0.0	1	0
13	709038	916	103965	30-34	m	16	5117	0	0.0	1	0
14	709040	916	103965	30-34	m	16	5120	0	0.0	1	0
15	709059	916	103968	30-34	m	20	14669	7	10.28000021	1	1
16	709105	916	103976	30-34	m	28	1241	0	0.0	1	1
17	709115	916	103978	30-34	m	30	2305	1	0.569999993	1	0
18	709124	916	103979	30-34	m	31	1024	0	0.0	1	1
19	709179	916	103988	35-39	m	15	4627	1	1.690000057	1	0
20	709183	916	103989	35-39	m	16	21026	4	4.630000114	2	1

- *run_quality_checks_staaging_table.py* - Data Quality Cheks

run_quality_checks_staging_table.py

scripts > run_quality_checks_staging_table.py > ...

```python
import sys
import os
import logging
from google.cloud import bigquery
from dotenv import load_dotenv


# setup logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("logs/run_quality_checks_staging_table.log"),
        logging.StreamHandler(sys.stdout)
    ]
)


# load environment
load_dotenv("/opt/airflow/AD-CAMPAIGN-DATAFLOW/.env")

BQ_PROJECT = os.getenv("BQ_PROJECT")
BQ_DATASET = os.getenv("BQ_DATASET")
BQ_STG_TABLE = os.getenv("BQ_STG_TABLE")



# create BigQuery client
def get_bq_client():
    try:
        client = bigquery.Client(project=BQ_PROJECT)
        logging.info(f"BigQuery client created for project: {BQ_PROJECT}")
        return client
    except Exception as e:
        logging.error(f"Failed to create BigQuery client: {e}")
        sys.exit(1)



# run a query and return the count
def run_bq_query(client, query: str) -> int:
    try:
        result = client.query(query).to_dataframe()
        return int(result.iloc[0, 0])
    except Exception as e:
        logging.error(f"Query failed: {e}\nQuery: {query}")
        sys.exit(1)

```

- Results:

```
[2025-10-12 14:41:54] INFO - 2025-10-12 11:41:54,795 [INFO] No NULL values in stg_ad_logs_tbl.ad_id source=task.stdout
[2025-10-12 14:41:56] INFO - 2025-10-12 11:41:56,406 [INFO] No NULL values in stg_ad_logs_tbl.impressions source=task.stdout
[2025-10-12 14:41:57] INFO - 2025-10-12 11:41:57,938 [INFO] No NULL values in stg_ad_logs_tbl.clicks source=task.stdout
[2025-10-12 14:41:59] INFO - 2025-10-12 11:41:59,328 [INFO] No duplicate rows in stg_ad_logs_tbl based on ad_id, xyz_campaign_id, fb_campaign_id source=task.stdout
[2025-10-12 14:42:00] INFO - 2025-10-12 11:42:00,862 [INFO] All stg_ad_logs_tbl.impressions values are >= 0 source=task.stdout
[2025-10-12 14:42:02] INFO - 2025-10-12 11:42:02,457 [INFO] All stg_ad_logs_tbl.clicks values are >= 0 source=task.stdout
[2025-10-12 14:42:04] INFO - 2025-10-12 11:42:04,043 [INFO] All stg_ad_logs_tbl.spent values are >= 0 source=task.stdout
[2025-10-12 14:42:04] INFO - 2025-10-12 11:42:04,043 [INFO] All quality checks PASSED. source=task.stdout
[2025-10-12 14:42:04] INFO - Done. Returned value was: None source=airflow.task.operators.airflow.providers.standard.operators.python.PythonOperator loc=python.py:218
```

- *daily_ad_logs.sql* – create materialized view on dataset

daily_ad_logs_view.sql

dbt_ads_project > models > staging > daily_ad_logs_view.sql

```sql
WITH raw AS (
    SELECT *
    FROM {{ source('staging_source', 'stg_ad_logs_tbl') }}
)

SELECT
    ad_id,
    xyz_campaign_id,
    fb_campaign_id,
    age,
    LOWER(gender) AS gender,
    COALESCE(CAST(interest AS STRING), 'NA') AS interest,
    impressions,
    clicks,
    spent,
    total_conversion,
    approved_conversion
FROM raw
```

- Results:

daily_a... iew

daily_ad_logs_view Query Open in Share Copy Snapshot Delete Export

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

Row	ad_id	xyz_campaig...	fb_campaign...	age	gender	interest	impressions	clicks	spent	total_conver...	approved_co...
1	708746	916	103916	30-34	m	15	7350	1	1.429999948	2	1
2	708749	916	103917	30-34	m	16	17861	2	1.820000023	2	0
3	708771	916	103920	30-34	m	20	693	0	0.0	1	0
4	708818	916	103928	30-34	m	28	4133	1	1.289999962	1	1
5	708815	916	103928	30-34	m	28	4259	1	1.25	1	0
6	708820	916	103929	30-34	m	29	1915	0	0.0	1	1
7	708889	916	103940	30-34	m	15	15615	3	4.769999981	1	0
8	708895	916	103941	30-34	m	16	10951	1	1.269999981	1	1
9	708953	916	103951	30-34	m	27	2355	1	1.5	1	0
10	708958	916	103952	30-34	m	28	9502	3	3.159999967	1	0
11	708979	916	103955	30-34	m	31	1224	0	0.0	1	0
12	709023	916	103962	30-34	m	7	735	0	0.0	1	0
13	709038	916	103965	30-34	m	16	5117	0	0.0	1	0
14	709040	916	103965	30-34	m	16	5120	0	0.0	1	0
15	709059	916	103968	30-34	m	20	14669	7	10.28000021	1	1
16	709105	916	103976	30-34	m	28	1241	0	0.0	1	1

- *campaign_daily_metrics_view.sql* – aggregate data and group by company id materialized view

campaign_daily_metrics_view.sql

dbt_ads_project > models > staging > campaign_daily_metrics_view.sql

```sql
SELECT
    xyz_campaign_id,
    COUNT(DISTINCT ad_id) AS total_ads,
    SUM(impressions) AS total_impressions,
    SUM(clicks) AS total_clicks,
    SUM(spent) AS total_spent,
    SUM(total_conversion) AS total_conversion,
    SUM(approved_conversion) AS total_approved_conversion,
    ROUND(CAST(SUM(clicks) AS NUMERIC) / NULLIF(SUM(impressions), 0), 4) AS ctr
FROM {{ ref('daily_ad_logs_view') }}
GROUP BY xyz_campaign_id

```

- Results:

campai... iew

campaign_daily_metrics_view

Query | Open in | Share | Copy | Snapshot | Delete | Export

Schema | Details | Preview | Table Explorer | Preview | Insights | Lineage | Data Profile | Data Quality

Row	xyz_campaig...	total_ads	total_impres...	total_clicks	total_spent	total_conver...	total_approv...	ctr
1	1178	625	204823716	36068	55662.14995...	2669	872	0.0002
2	916	54	482925	113	149.7100006...	58	24	0.0002
3	936	464	8128187	1984	2893.369998...	537	183	0.0002

- *Staging_to_datamodel.py* – load data from staging to data model table

staging_to_datamodel.py ×

scripts > staging_to_datamodel.py > ...

```python
import os
import sys
import logging
import pandas as pd
from google.cloud import bigquery
from dotenv import load_dotenv

# setup logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("logs/staging_to_datamodel.log"),
        logging.StreamHandler(sys.stdout)
        ]
)


# load environment
load_dotenv("/opt/airflow/AD-CAMPAIGN-DATAFLOW/.env")

BQ_PROJECT = os.getenv("BQ_PROJECT")
BQ_DATASET = os.getenv("BQ_DATASET")
BQ_STG_TABLE = os.getenv("BQ_STG_TABLE")
BQ_DM_TABLE = os.getenv("BQ_DM_TABLE")



# create BigQuery client
def get_bq_client():
    try:
        client = bigquery.Client(project=BQ_PROJECT)
        logging.info(f"Connected to BigQuery project: {BQ_PROJECT}")
        return client
    except Exception as e:
        logging.error(f"Failed to create BigQuery client: {e}")
        sys.exit(1)



# read data from staging table
def read_staging_data(client: bigquery.Client) -> pd.DataFrame:
    query = f"SELECT * FROM `{BQ_PROJECT}.{BQ_DATASET}.{BQ_STG_TABLE}`"
    try:
        df = client.query(query).to_dataframe()
        logging.info(f"Loaded {len(df)} rows from {BQ_STG_TABLE}.")
        return df
    except Exception as e:
```

- Result:

dm_ad_...tbl

dm_ad_logs_tbl Query Open in Share Copy Snapshot Delete Export

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

Row	ad_id	campaign_id	fb_campaign...	age	gender	interest	impressions	clicks	spent	total_conver...	approved_co...	age_mid	click_rate	spent_rate	reach
1	708746	916	103916	30-34	Male	15	7350	1	1.429999948	2	1	32.0	0.013605442...	142.9999948	0.000136054...
2	708749	916	103917	30-34	Male	16	17861	2	1.820000023	2	0	32.0	0.011197581...	91.00000115	0.000111975...
3	708771	916	103920	30-34	Male	20	693	0	0.0	1	0	32.0	0.0	0.0	0.0
4	708815	916	103928	30-34	Male	28	4259	1	1.25	1	0	32.0	0.023479690...	125.0	0.000234796...
5	708818	916	103928	30-34	Male	28	4133	1	1.289999962	1	1	32.0	0.024195499...	128.9999962	0.000241954...
6	708820	916	103929	30-34	Male	29	1915	0	0.0	1	1	32.0	0.0	0.0	0.0
7	708889	916	103940	30-34	Male	15	15615	3	4.769999981	1	0	32.0	0.019212295...	158.9999993...	0.000192122...
8	708895	916	103941	30-34	Male	16	10951	1	1.269999981	1	1	32.0	0.009131586...	126.9999981	9.131586156...
9	708953	916	103951	30-34	Male	27	2355	1	1.5	1	0	32.0	0.042462845...	150.0	0.000424628...
10	708958	916	103952	30-34	Male	28	9502	3	3.159999967	1	0	32.0	0.031572300...	105.3333322...	0.000315723...
11	708979	916	103955	30-34	Male	31	1224	0	0.0	1	0	32.0	0.0	0.0	0.0
12	709023	916	103962	30-34	Male	7	735	0	0.0	1	0	32.0	0.0	0.0	0.0
13	709038	916	103965	30-34	Male	16	5117	0	0.0	1	0	32.0	0.0	0.0	0.0
14	709040	916	103965	30-34	Male	16	5120	0	0.0	1	0	32.0	0.0	0.0	0.0
15	709059	916	103968	30-34	Male	20	14669	7	10.28000021	1	1	32.0	0.047719680...	146.8571458...	0.000477196...
16	709105	916	103976	30-34	Male	28	1241	0	0.0	1	1	32.0	0.0	0.0	0.0
17	709115	916	103978	30-34	Male	30	2305	1	0.569999993	1	0	32.0	0.043383947...	56.99999930...	0.000433839...
18	709124	916	103979	30-34	Male	31	1024	0	0.0	1	1	32.0	0.0	0.0	0.0
19	709179	916	103988	35-39	Male	15	4627	1	1.690000057	1	0	37.0	0.021612275...	169.0000057	0.000216122...

- *campaign_metrics_view.sql* – aggregate data and group by company id materialized view for the whole data

campaign_metrics_view.sql

dbt_ads_project > models > data_model > campaign_metrics_view.sql

```sql
WITH base AS (
    SELECT
        campaign_id,
        fb_campaign_id,
        age,
        gender,
        interest,
        SUM(impressions) AS total_impressions,
        SUM(clicks) AS total_clicks,
        SUM(spent) AS total_spent,
        SUM(total_conversion) AS total_conversion,
        SUM(approved_conversion) AS approved_conversion
    FROM {{ source('datamodel_source','dm_ad_logs_tbl') }}
    GROUP BY 1, 2, 3, 4, 5
),

metrics AS (
    SELECT
        campaign_id,
        fb_campaign_id,
        age,
        gender,
        interest,
        total_impressions,
        total_clicks,
        total_spent,
        total_conversion,
        approved_conversion,
        CASE WHEN total_impressions > 0 THEN CAST(total_clicks AS FLOAT64) / total_impressions ELSE 0 END AS click_rate,
        CASE WHEN total_clicks > 0 THEN CAST(total_spent AS FLOAT64) / total_clicks ELSE 0 END AS spent_rate,
        CASE WHEN total_impressions > 0 THEN CAST(total_clicks AS FLOAT64) / total_impressions ELSE 0 END AS reach
    FROM base
)

SELECT * FROM metrics
```

- Results:

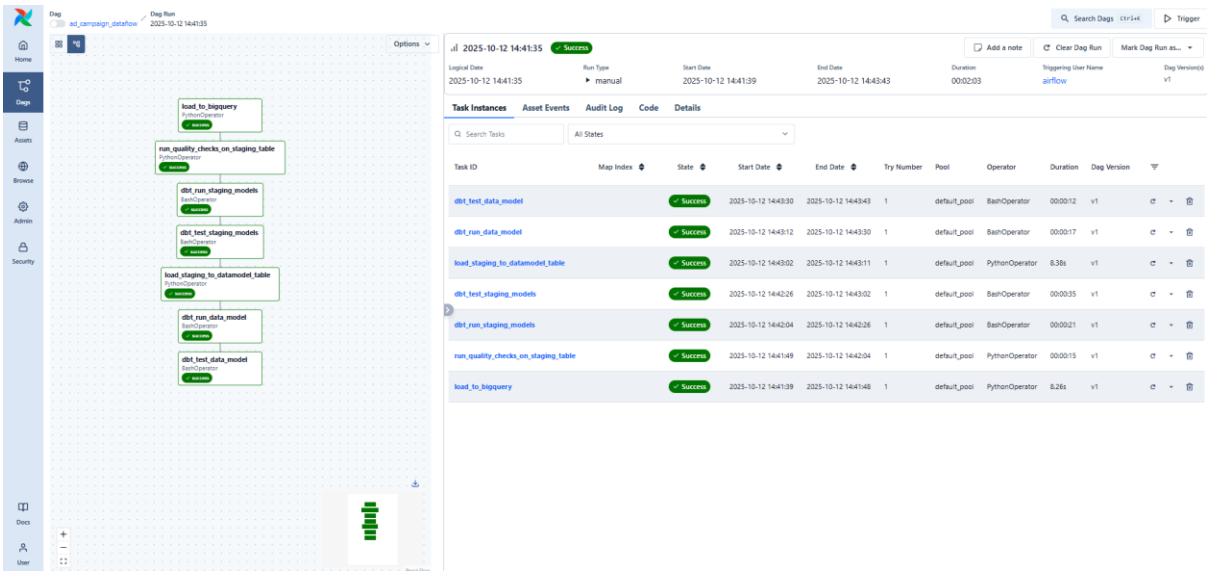
campai... iew

campaign_metrics_view Query Open in Share Copy Snapshot Delete Export

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

Row	campaign_id	fb_campaign...	age	gender	interest	total_impres...	total_clicks	total_spent	total_conver...	approved_co...	click_rate	spent_rate	reach
1	936	108672	30-34	Male	36	1752	0	0.0	2	1	0.0	0.0	0.0
2	936	123662	30-34	Female	26	1083	0	0.0	2	1	0.0	0.0	0.0
3	936	109473	30-34	Male	29	3783	0	0.0	2	0	0.0	0.0	0.0
4	936	110947	30-34	Female	30	493	0	0.0	1	0	0.0	0.0	0.0
5	916	104270	30-34	Female	63	1185	0	0.0	1	0	0.0	0.0	0.0
6	936	115866	30-34	Male	31	2214	0	0.0	1	0	0.0	0.0	0.0
7	936	116471	30-34	Female	20	1984	0	0.0	1	0	0.0	0.0	0.0
8	936	116589	30-34	Male	15	4827	0	0.0	1	0	0.0	0.0	0.0
9	936	109451	30-34	Male	15	2522	0	0.0	1	0	0.0	0.0	0.0
10	936	115551	30-34	Male	2	545	0	0.0	1	1	0.0	0.0	0.0
11	936	123558	30-34	Male	27	199	0	0.0	1	0	0.0	0.0	0.0
12	916	103920	30-34	Male	20	693	0	0.0	1	0	0.0	0.0	0.0
13	936	109738	30-34	Female	64	1539	0	0.0	1	0	0.0	0.0	0.0
14	936	115876	30-34	Male	30	1371	0	0.0	1	1	0.0	0.0	0.0

- Airflow Orchestration DAG



7) Summary

What started as a local ETL prototype (PostgreSQL + manual execution) has evolved into a scalable, automated, cloud-based data pipeline with,

- GCP BigQuery for analytical warehousing
- dbt for modular transformations and testing
- Dockerized Airflow for orchestration
- Automated daily scheduling.

This setup demonstrates a modern ELT architecture ready for analytics and can easily extend to other data sources and ad platforms.